

MedAsistan

Teknik Rapor

Proje konumu

Sağlık (HealthTech) kategorisinde geliştirilen MedAsistan, hastanın ilaç ve tedavi sürecini daha düzenli yönetmesini; doktorun ise güncel verilerle hastayı takip etmesini sağlayan akıllı hasta takip sistemidir.

Kategori	Sağlık / Hasta Takip Süreci
Ana problem	Süreç tıkanıklıkları, veri kopukluğu ve ilaç uyum takibi
Teknoloji	Next.js, React, TypeScript, Supabase, Yapay Zeka
Tarih	17 Mayıs 2026

Takım üyeleri:

1. Mahir Enes Akpınar
2. Eyup Ensar Toğuşlu
3. Abdullah Amin

İçindekiler

1. Proje Özeti ve Problem
2. Kullanılan Teknolojiler
3. Sistem Mimarisi
4. Yarışma Süreci ve Geliştirme Yaklaşımı
5. Ana Özellikler
6. Veri Güvenliği
7. Demo Senaryosu
8. Gelecek Planları
9. Sonuç

1. Proje Özeti ve Problem

MedAsistan, Base41 Hackathon kapsamında **Sağlık (HealthTech)** kategorisi için geliştirilen bir hasta takip ve ilaç uyum sistemidir. Proje, özellikle **hasta takip sürecindeki tıkanıklıklar, veri kopukluğu ve tedavi sürecinde hastanın bir sonraki adımı net görememesi** problemlerine odaklanır.

Sağlık süreçlerinde hastanın ilaç kullanım bilgisi çoğu zaman dağınık, geç işlenen veya yalnızca hastanın beyanına bağlı kalan bir veri halindedir. Bu durum doktorun tedavi sürecini güncel verilerle takip etmesini zorlaştırır. Hasta tarafında ise ilaç saatlerini unutma, son kullanma tarihlerini takip edememe, reçete veya kullanım planını düzenli yönetememe gibi problemler oluşur.

MedAsistan bu problemi iki taraflı bir yaklaşımla çözer:

- **Hasta tarafı:** Hasta kendi ilaçlarını ekleyebilir, doz saatlerini takip edebilir, ilaç alımlarını işaretleyebilir, son kullanma tarihi yaklaşan ilaçları görebilir ve doktor davetlerini kabul edebilir.
- **Doktor tarafı:** Doktor hastalarını davet edebilir, hastanın aktif ilaçlarını ve uyum oranını görebilir, ilaç reçete edebilir, kullanım saatlerini düzenleyebilir ve hasta hakkında özel notlar tutabilir.

Bu yönüyle MedAsistan yalnızca bir hatırlatma uygulaması değil, hastanın sürecini daha otonom yönettiği, doktorun ise anlık ve anlamlı verilerle takip yapabildiği **akıllı hasta takip sistemi** olarak tasarlanmıştır.

2. Kullanılan Teknolojiler

Frontend

Projede frontend tarafında **Next.js 16 App Router, React 19, TypeScript** ve **Tailwind CSS v4** kullanılmıştır.

Next.js App Router yapısı, sayfa bazlı routing, server component desteği ve modern dosya organizasyonu sayesinde projenin hızlı geliştirilmesini sağlamıştır. React 19 ile bileşen tabanlı, yeniden kullanılabilir ve etkileşimli bir arayüz kurulmuştur. TypeScript tercih edilerek veri modelleri ve bileşenler daha güvenli hale getirilmiş, olası tip hatalarının geliştirme aşamasında yakalanması hedeflenmiştir.

Tailwind CSS v4 ve özel OKLCH renk tokenları ile sade, erişilebilir ve sağlık uygulamasına uygun bir arayüz tasarlanmıştır. Hasta panelinde daha sıcak mavi tonlar, doktor panelinde ise daha klinik ve profesyonel yeşil tonlar kullanılmıştır. Arayüzde hızlı anlaşılabilirlik için lucide-react ikonları, kullanıcı geri bildirimleri için sonner toast sistemi tercih edilmiştir.

Backend ve Veritabanı

Backend ve veritabanı tarafında **Supabase Auth** ve **Supabase PostgreSQL** kullanılmıştır. Supabase Auth, kullanıcı kayıt/giriş işlemlerini ve oturum yönetimini sağlamaktadır. Uygulama içinde iki temel rol bulunmaktadır:

- Hasta
- Doktor

Veritabanında temel olarak şu tablolar kullanılmaktadır:

- profiles
- medicines
- schedules
- dose_logs
- notifications
- doctor_profiles
- doctor_patients

- doctor_notes

Hasta verileri hassas sađlık verisi niteliğinde olduđu için Supabase tarafında **Row Level Security (RLS)** yaklaşımı kritik bir güvenlik katmanı olarak kullanılmıştır. RLS politikaları sayesinde kullanıcılar yalnızca kendi ilaçlarını, doz kayıtlarını, bildirimlerini ve profillerini yönetebilir. Doktor tarafında ise doktorun yalnızca aktif bağlantısı olan hastaların verilerini görebilmesi hedeflenmiştir.

Bu yapı, sađlık verilerinde yetkisiz erişimi önlemek ve doktor-hasta ilişkisinin veritabanı seviyesinde sınırlandırılmasını sađlamak için tercih edilmiştir.

Yapay Zeka ve Sponsor Entegrasyonu

Projede yapay zeka katmanı, ilaç ekleme sürecini kolaylaştırmak için kullanılmaktadır. Hasta, ilacın kutu görselini ve son kullanma tarihi görselini yüklediğinde sistem bu görselleri analiz ederek şu bilgileri otomatik çıkarmaya çalışır:

- İlaç adı
- Etken madde
- Doz bilgisi
- Son kullanma tarihi
- Tahmini kutu/adet bilgisi

Mevcut prototipte bu analiz hattı `POST /api/analyze-medicine` API rotası üzerinden çalışmakta ve görsel analiz için Gemini 2.0 Flash modeli kullanılmaktadır. Yarışma sponsor entegrasyonu kapsamında bu AI analiz katmanı Puq.ai entegrasyonuna uygun şekilde konumlandırılmıştır. Yani görselden veri çıkarma akışı tek bir backend API rotasında toplandığı için model sağlayıcısı değiştirildiğinde frontend akışı bozulmadan Puq.ai servisine yönlendirme yapılabilir.

Bu yaklaşımın amacı, yapay zekayı projeye yalnızca dekoratif bir özellik olarak eklemek değil, doğrudan problem çözümünün parçası haline getirmektir. İlaç bilgilerinin manuel girilmesi özellikle yaşlı kullanıcılar veya teknolojiye uzak kullanıcılar için hata üretmeye açıktır. Görsel analiz bu yükü azaltarak hasta deneyimini hızlandırır ve veri giriş kalitesini artırır.

Puq.ai Günlük Hatırlatma Workflow'u

Puq.ai tarafında ayrıca ilaç hatırlatma sürecini otomatikleştiren bir workflow hazırlanmıştır. Bu workflow her gün belirli bir zamanlayıcı ile tetiklenir, Supabase veritabanından ilgili hatırlatma kayıtlarını alır, kayıtları tek tek işler ve kullanıcıya Gmail entegrasyonu üzerinden e-posta bilgilendirmesi gönderir.

Bu akış, MedAsistan'ın yalnızca uygulama içi takip ekranlarından oluşmadığını; dış otomasyon servisleriyle zaman bazlı bildirim süreçlerini de yürütebilecek bir yapıya sahip olduğunu gösterir. Hackathon kapsamında bu workflow, Puq.ai sponsor entegrasyonunun projeye somut bir süreç otomasyonu olarak dahil edildiğini ortaya koymaktadır.

Adım	Servis	Görev
1. Every Day	Puq.ai Schedule	Workflow'u günlük olarak tetikler.
2. Select Data	Supabase	İlgili ilaç, doz veya hatırlatma kayıtlarını seçer.
3. Loop on Items	Puq.ai Loop	Gelen kayıtları tek tek işler.
4. Send Email	Gmail	Kullanıcıya hatırlatma veya bilgilendirme e-postası gönderir.

```
Puq.ai Schedule
-> Supabase Select Data
-> Loop on Items
-> Gmail Send Email
```

İlerleyen aşamada bu workflow aynı mantıkla gerçek push bildirimleri, SMS/WhatsApp hatırlatmaları veya doktor-hasta uyarı otomasyonları için genişletilebilir.

3. Sistem Mimarisi

MedAsistan üç ana katmandan oluşur:

- 1. Frontend katmanı:** Next.js sayfaları ve React bileşenleri.
- 2. Backend/API katmanı:** Next.js route handlers ve server actions.
- 3. Veri katmanı:** Supabase Auth, PostgreSQL tabloları ve RLS politikaları.

Genel akış şu şekildedir:

```
Kullanıcı Arayüzü
-> Next.js Client/Server Components
-> Supabase Client veya Next.js API Route
-> Supabase Auth + PostgreSQL
-> RLS politikaları ile yetki kontrolü
```

Önemli API ve server action noktaları:

- **POST /api/analyze-medicine:** İlaç fotoğraflarını analiz eder.
- **DELETE /api/medicines/[id]:** İlaç, kullanım planı ve doz kayıtlarını güvenli sırayla siler.
- **findPatientByEmail:** Doktorun hastayı e-posta ile bulmasını sağlar.
- **sendDoctorInvite:** Doktor-hasta davet ilişkisi oluşturur.
- **Puq.ai workflow:** Günlük tetiklenen Supabase veri seçimi, kayıt döngüsü ve Gmail e-posta gönderimi ile hatırlatma otomasyonunu yürütür.

Bu mimari, frontend ile veritabanı arasındaki iş mantığını kontrollü şekilde ayırır. Basit hasta işlemleri Supabase client üzerinden yapılırken, daha kritik işlemler özel backend API rotalarında ele alınmıştır.

4. Yarışma Süreci ve Geliştirme Yaklaşımı

Geliştirme sürecinde iş bölümü temel olarak üç eksene ayrılmıştır:

- **Frontend ve kullanıcı deneyimi:** Hasta paneli, doktor paneli, responsive layout, form akışları ve dashboard ekranları.
- **Backend ve veritabanı:** Supabase tabloları, RLS politikaları, kullanıcı rolleri, doktor-hasta ilişkileri ve API rotaları.
- **Yapay zeka ve sunum:** İlaç fotoğrafı analiz akışı, proje anlatımı, demo senaryosu ve teknik raporlama.

Süreçte en önemli teknik zorluklardan biri ilişkili verilerin silinmesi sırasında yaşanmıştır. Bir ilaç silinmek istendiğinde medicines tablosuna bağlı schedules, schedules tablosuna bağlı olarak da dose_logs kayıtları bulunduğu için Supabase tarafında foreign key çakışması oluşmuştur. İlk yaklaşımda frontend üzerinden önce planları, sonra ilacı silmek yeterli olmamıştır.

Bu problem için özel bir backend API rotası yazılmıştır:

```
DELETE /api/medicines/[id]
```

Bu rota şu sırayla çalışır:

1. Kullanıcının oturumunu doğrular.
2. Silinmek istenen ilacın gerçekten kullanıcıya ait olup olmadığını kontrol eder.
3. İlgili kullanım planlarını bulur.
4. Önce bu planlara bağlı dose_logs kayıtlarını siler.
5. Sonra schedules kayıtlarını siler.
6. En son medicines kaydını siler.

Bu çözüm, sistemin veri bütünlüğünü korurken kullanıcı tarafında tek tıklamayla güvenli silme deneyimi sunmasını sağlamıştır.

Zaman yönetiminde öncelik canlı demo için uçtan uca çalışan çekirdek akışlara verilmiştir. Önce hasta ilaç ekleme, doz takip ve envanter akışları tamamlanmış; ardından doktor davet, doktor paneli ve hasta detay ekranları geliştirilmiştir. Son aşamada yarışma sunumuna uygun olarak teknik rapor, proje özeti ve demo senaryosu hazırlanmıştır.

5. Ana Özellikler

Hasta Özellikleri

- E-posta/şifre ile kayıt ve giriş.
- Hasta veya doktor rolü seçimi.
- İlaç ekleme.
- Fotoğrafla ilaç bilgisi çıkarma.
- Manuel ilaç bilgisi girişi.
- Doz saatleri belirleme.
- Günlük dozları işaretleme.
- Haftalık doz takvimi görme.
- Envanterde ilaç arama ve filtreleme.
- İlaç düzenleme.
- İlaç silme.
- Son kullanma tarihi uyarıları.
- Doktor davetlerini kabul veya reddetme.
- Bildirimleri okundu olarak işaretleme.

Doktor Özellikleri

- Doktor onboarding ekranı.
- Uzmanlık, hastane, lisans numarası ve biyografi bilgisi girişi.
- Hasta davet etme.
- Bekleyen davetleri takip etme.
- Aktif hastaları görüntüleme.
- Hasta detayına girme.
- Hastanın ilaçlarını ve doz uyumunu görme.

- Hastaya ilaç reçete etme.
- Hastanın ilaçlarını düzenleme.
- Doktor notu ekleme.
- Hasta uyum oranlarını izleme.

6. Veri Güvenliği

MedAsistan sağlık verisiyle çalıştığı için güvenlik tasarımının merkezindedir.

Kullanılan güvenlik yaklaşımları:

- Supabase Auth ile oturum yönetimi.
- Protected route middleware.
- Doktor paneli için rol kontrolü.
- Supabase RLS politikaları.
- Kullanıcı sahipliği kontrolü.
- Doktor-hasta ilişkisinin doctor_patients tablosu üzerinden doğrulanması.
- Kritik silme işlemlerinin server-side API üzerinden yapılması.
- Service role key'in yalnızca server-side kullanılması.

Bu yapı ile hastanın verisi varsayılan olarak yalnızca hastaya aittir. Doktorun erişimi ise hasta daveti kabul ettikten sonra aktif hale gelir.

7. Demo Senaryosu

Canlı demo için önerilen akış:

1. Hasta hesabıyla giriş yapılır.
2. İlaç ekleme ekranı açılır.
3. Fotoğrafla veya manuel olarak ilaç eklenir.
4. Dashboard'da bugünkü dozlar gösterilir.
5. Bir doz alındı olarak işaretlenir.
6. Envanter ekranında ilaç listesi, filtreler ve son kullanma tarihi gösterilir.
7. Doktor hesabına geçilir.
8. Doktor panelinde hasta listesi ve uyum oranları gösterilir.
9. Doktor bir hastanın detayına girer.
10. Hastaya yeni ilaç reçete eder veya doktor notu ekler.

Bu senaryo, hem hasta hem doktor tarafının çalıştığını ve sistemin yalnızca statik bir arayüz olmadığını gösterir.

8. Gelecek Planları

Projeyi canlıya alırken planlanan geliştirmeler:

- Puq.ai sponsor entegrasyonunun üretim seviyesinde tamamlanması.
- Gerçek push bildirimleri.
- SMS veya WhatsApp hatırlatmaları.
- Doktor tarafından dijital reçete onayı.

- İlaç fotoğraflarının Supabase Storage üzerinde saklanması.
- Doktor profil düzenleme ekranı.
- Hasta yakınları için kontrollü paylaşım modu.
- Eczane veya ilaç stok entegrasyonu.
- Doz uyum raporlarının PDF olarak dışa aktarılması.
- Daha gelişmiş yapay zeka karar destek sistemi.
- Mobil uygulama veya PWA desteği.

9. Sonuç

MedAsistan, Sağlık kategorisinde hasta takip sürecindeki veri kopukluğu ve koordinasyon eksikliğine odaklanan bir çözümdür. Proje, hastanın günlük ilaç takibini kolaylaştırırken doktorun da tedavi sürecini daha görünür ve ölçülebilir hale getirmesini sağlar.

Teknik olarak Next.js, Supabase ve yapay zeka analiz katmanı üzerine kurulmuştur. RLS politikaları, doktor-hasta ilişki modeli ve güvenli API rotaları ile sağlık verisinin kontrollü yönetilmesi hedeflenmiştir.

Hackathon bağlamında proje, canlı demo sunabilecek seviyede çalışan bir hasta-doktor takip akışı sunmakta; gelecek geliştirmelerle gerçek bildirim servisleri, sponsor AI entegrasyonu ve daha gelişmiş klinik karar destek özellikleriyle büyütülebilecek bir temel oluşturmaktadır.